NO-CODE DEVELOPMENT · PROJECT PROPOSAL · SCOPING DELIVERABLE

Deadline Bot

A Telegram bot and a website that collect class deadlines from any source (typed messages, photos of slides, PDFs), save them to one shared list, answer `/deadlines` and send each student reminders when they want them.

Tool n8n **Channels** Telegram + website **Database** Supabase (Postgres) **AI** Claude Haiku 5.5 **Target level** 3

01 The need

> Students in my class lose track of deadlines because they're announced across slides, emails and out loud in class, so we waste time digging through everything or asking each other.

What we do today instead: reopen old slide decks, search our emails, and ask in the class group chat, where the answer is often "I think so". Planner apps exist, but someone has to type every deadline in by hand, and everyone keeps a separate list.

What the bot changes: any classmate sends what they saw or heard, as a typed message, a photo or a PDF. The AI extracts the deadline, and the whole class shares one up-to-date list, on Telegram and on a website.

02 The triggers

Main trigger: Telegram Trigger

It fires on every update sent to the bot: typed text, a photo, a PDF, a command (`/deadlines`, `/reminders`, `/help`) or a tap on one of the bot's buttons.

Website trigger: Webhook

The website (one HTML page on Vercel) sends every action to a single n8n webhook: `list`, `add`, `add_file` or `delete`. The page never talks to the database or the AI directly.

Testing on demand

I send the bot a message myself, anytime. I keep a fixed set of test inputs and resend the same ones after every change:

TEST INPUT	WHAT IT CHECKS
`Data Viz report due 24/10 at 23:59`	Clean case: the basic pipeline works end to end.
`the essay is due next Friday`	Vague case: "next Friday" becomes a real date; missing course is handled.
`hello`	No deadline: the bot replies "no deadline found" instead of saving garbage.
`/deadline (typo)`	Wrong command gets a helpful reply ("did you mean /deadlines?").
`the same message twice`	Idempotent: the second time it says "already recorded", no duplicate row.
`delete the Data Viz report`	The right deadline is found and the bot asks Yes / Cancel before deleting.
`1 sample PDF + 1 photo of a slide`	File download, then Claude reads every deadline in the file.
`a text file renamed fake.pdf`	Failure path: the user gets "I couldn't read that file" instead of silence.

Third trigger: Schedule Trigger

Runs every hour (Paris time). Each student chooses the hour and how often they are reminded, so the workflow checks every hour who is due a message. A Manual Trigger ("Run now") is wired next to it so I can test reminders immediately.

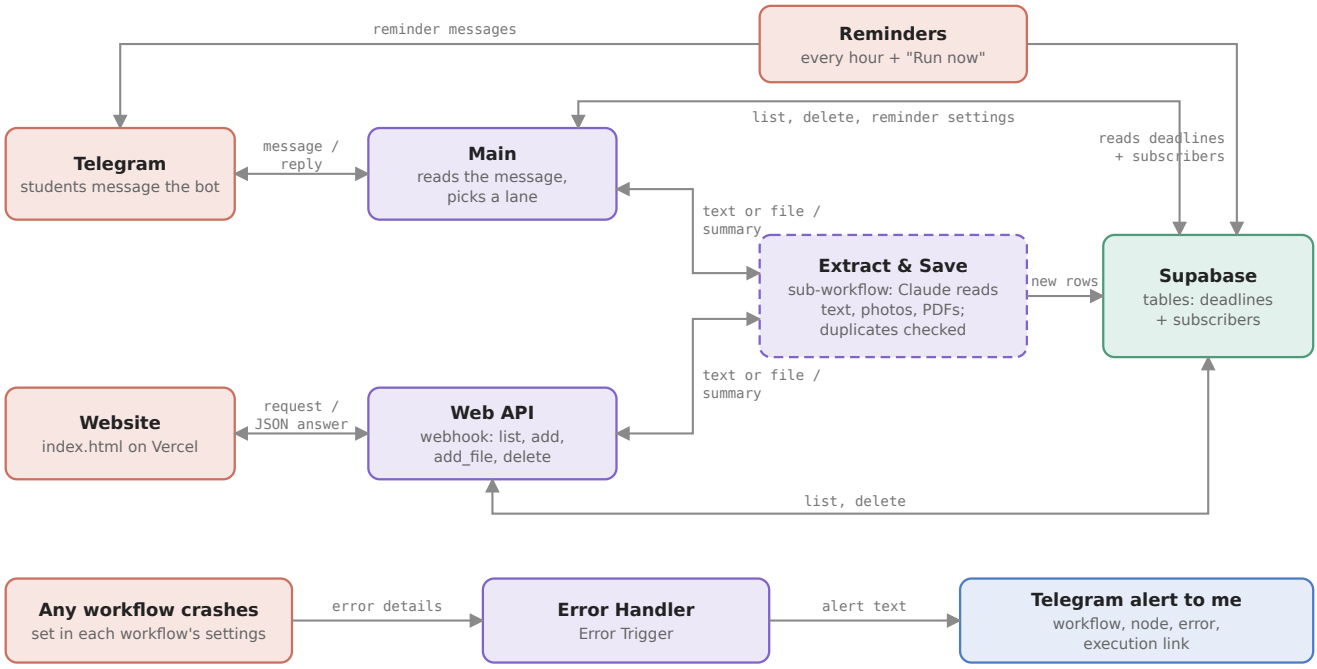
03 Architecture diagram

Five workflows. Grey labels on the arrows show what travels between blocks.

Trigger / input Logic (Switch, If, Set, Code, Merge) AI & external APIs Database Reply to the user Separate workflow / setting

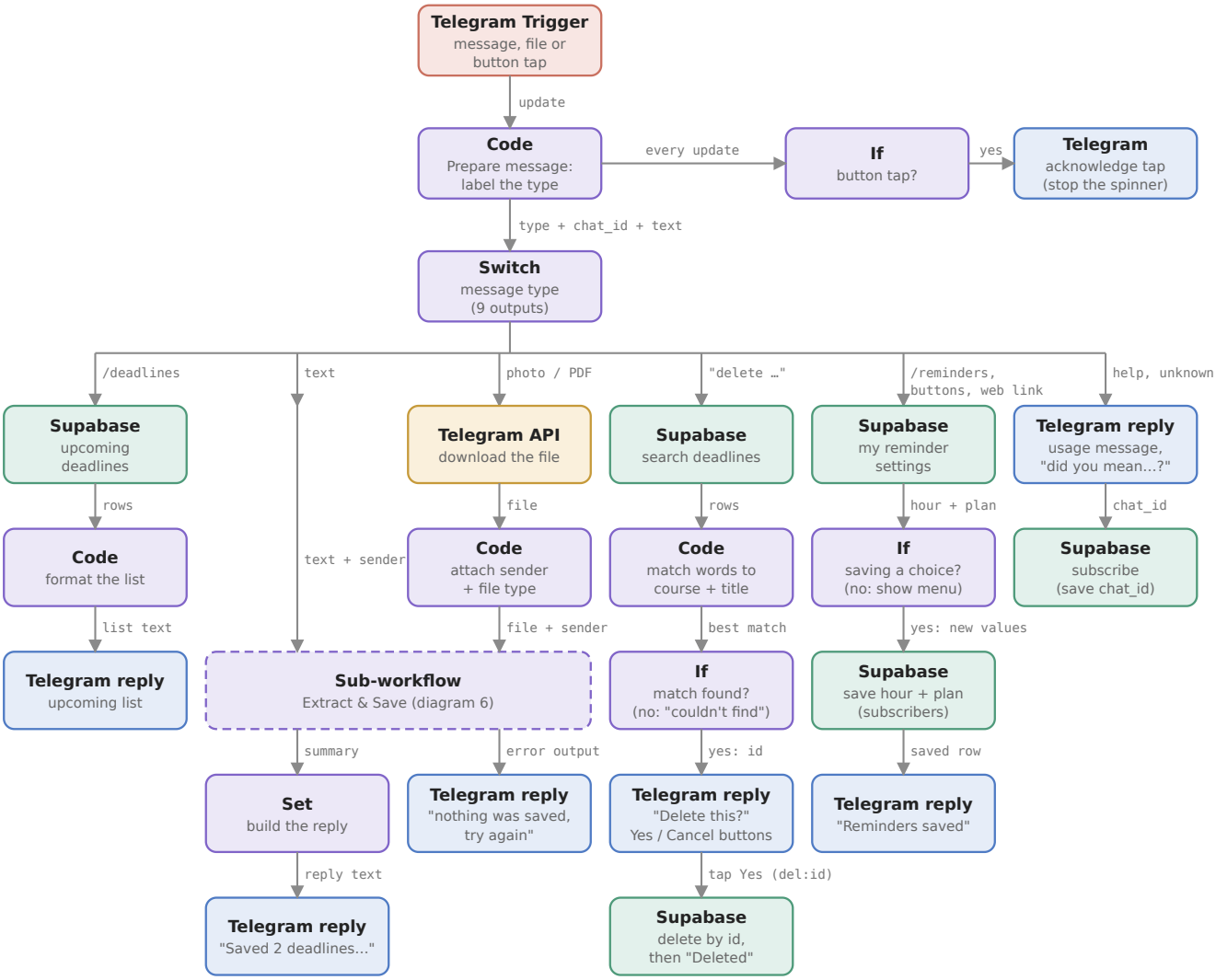
1 · Overview: how the five workflows connect

Telegram and the website are two doors into the same system. Both send new deadlines to the same sub-workflow and read the same Supabase tables.



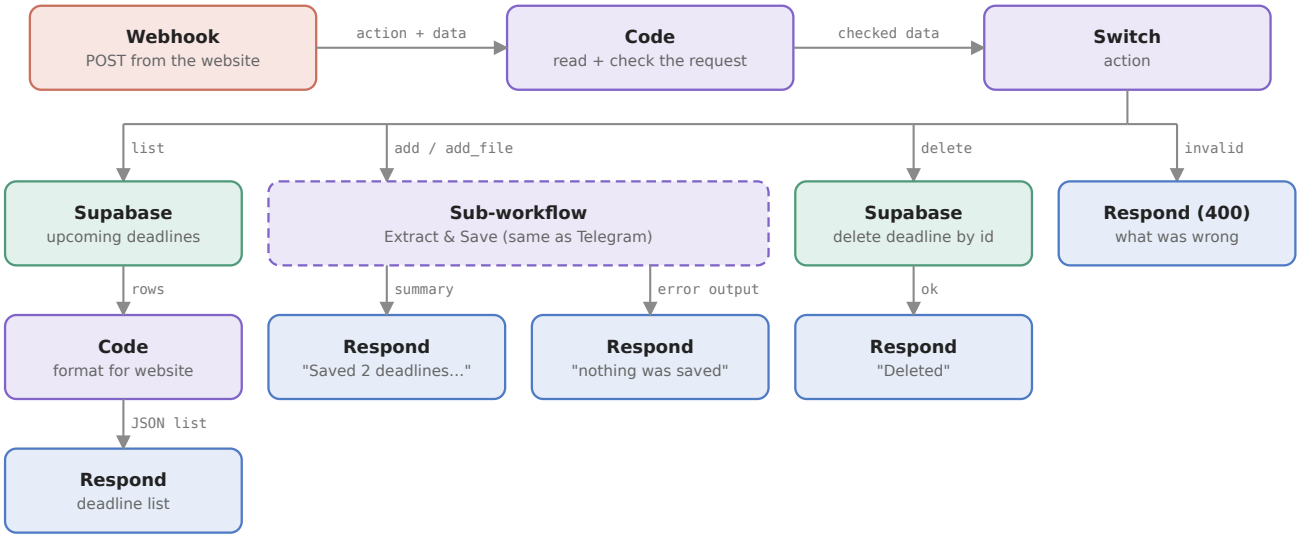
2 · Main workflow: receiving a Telegram message

A Code node labels each update, then one Switch sends it to its lane. Text and files both go to the Extract & Save sub-workflow. Button taps are also acknowledged in parallel so Telegram stops showing a spinner.



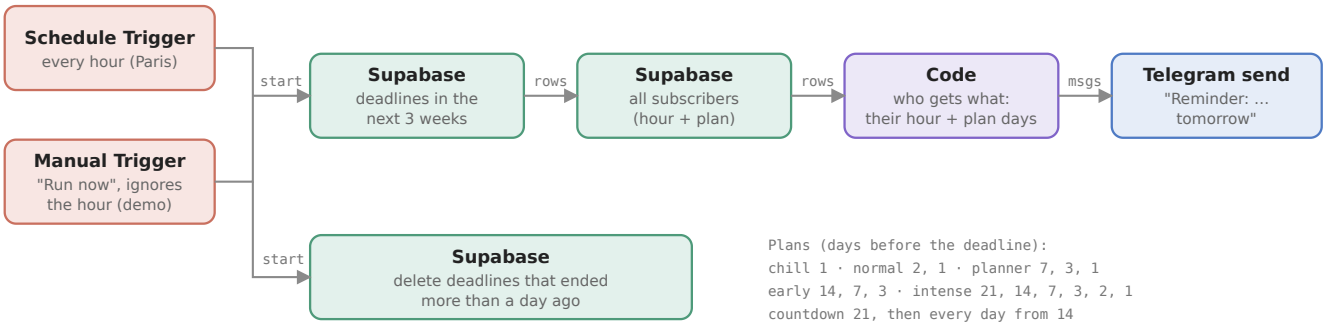
3 · Web API workflow: the website's backend

Every request names an action; a Switch routes it and a Respond node always answers. Adding uses the same sub-workflow as Telegram, so both give the same result.



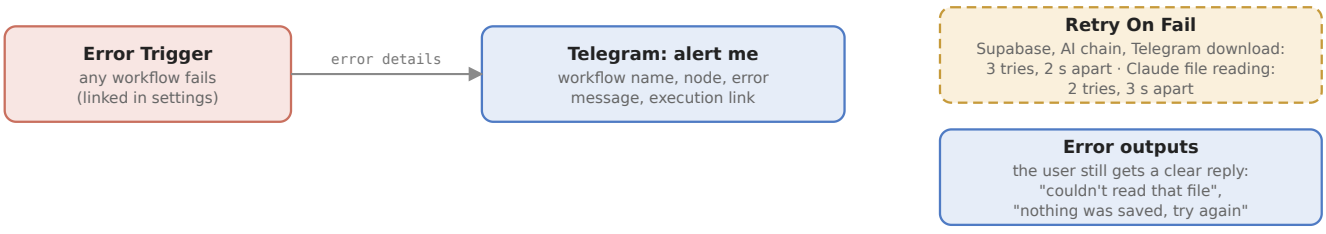
4 · Reminder workflow

Runs every hour. For each subscriber, the Code node checks two things: is it their chosen hour, and is a deadline exactly one of their plan's days away? It also cleans up old deadlines.



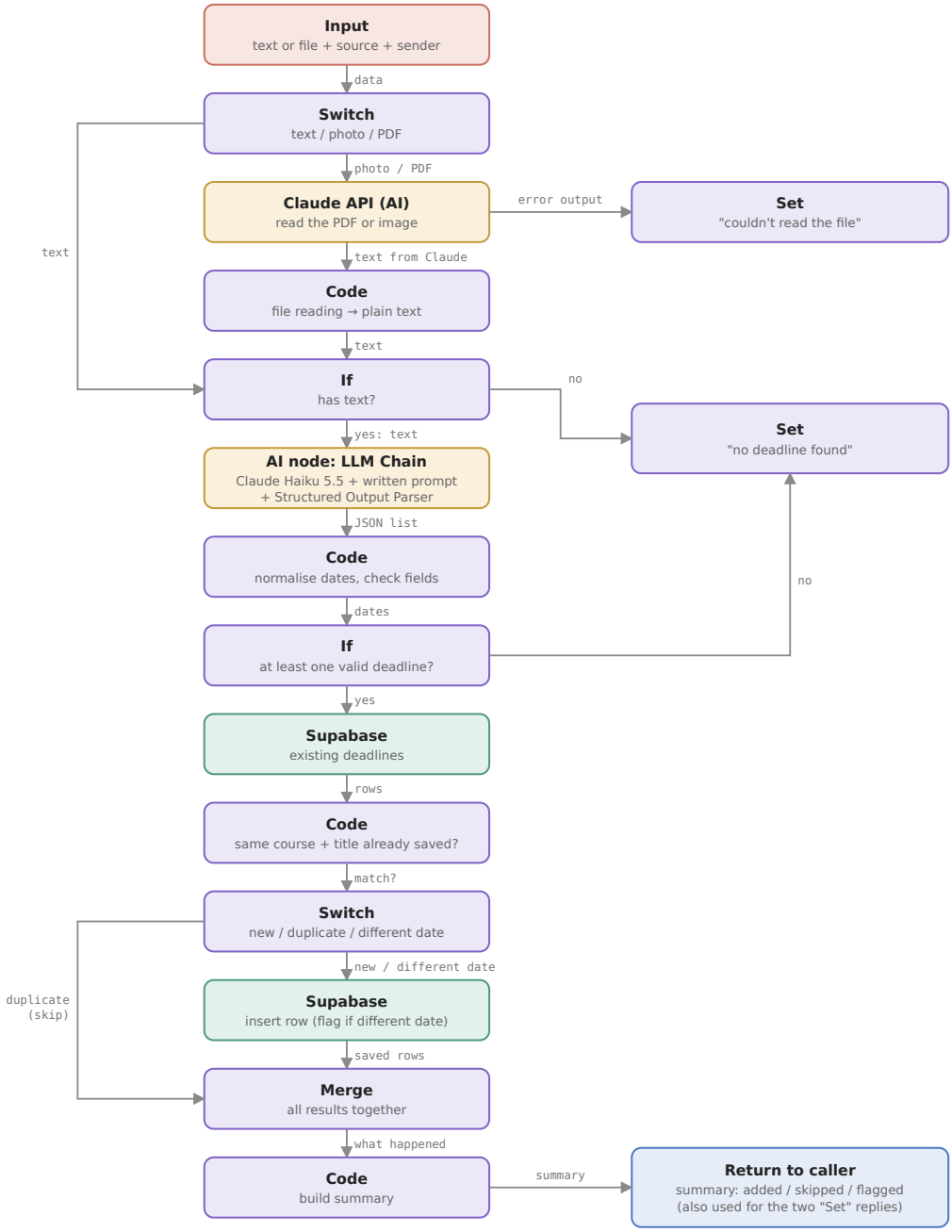
5 · Error handling

Three layers: retries on external calls, error outputs so the user always gets an answer, and an Error Workflow that alerts me when a run still fails.



6 · Sub-workflow: Extract & Save Deadlines

Called by Main and by the Web API. Files are turned into text first, then the AI returns structured JSON, a Code node cleans the dates, and each deadline is saved, skipped as a duplicate, or saved with a conflict flag.



04 Level and cut line

Target: Level 3. The project uses a sub-workflow shared by two channels, error handling with retries and an Error Workflow, multi-step logic, and stored state (duplicate and conflict detection, each student's reminder settings).

Build strategy: I first got a simple version running end to end (typed text → AI → database → reply), then added the Level 3 parts on top, so a working version was always ready for the demo.

What I cut

Voice notes. Typed messages, photos and PDFs already cover how deadlines reach us; a deadline said out loud in class can simply be typed. Speech-to-text would have added another paid API for little benefit.

Never cut: typed text → AI extraction → saved in the database → confirmation reply, plus `/deadlines` and the usage message for wrong input.

05 Minimum technical requirements

REQUIREMENT	WHERE IT'S USED
If or Set node	If: "button tap?", "match found?", "has text?", "at least one valid deadline?", "saving a choice?". Set: building the reply message, "no deadline found", "couldn't read the file".
Code node	Prepare message (labels each update), normalising dates ("next Friday" → 2026-10-23), checking duplicates, matching a deadline to delete, building reminders, formatting the `/deadlines` list.
Router or If/Switch	Switch on message type (9 outputs: list, text, file, delete, reminders…). Switch on new / duplicate / different date. Switch on text / photo / PDF. Switch on the website's action.
API or MCP connection	Telegram Bot API (messages, buttons, file download), Anthropic API (Claude reads PDFs and images), Supabase, and the Web API webhook for the website. All with stored credentials.
Message trigger	Telegram Trigger (plus a Webhook for the website).
Database	Supabase (Postgres): `deadlines` table (course, title, due date, source, added by, conflict flag) and `subscribers` table (chat id, reminder hour, reminder plan).
AI node	LLM Chain with Claude Haiku 5.5, a written prompt and a Structured Output Parser, extracting deadlines as JSON.
Sub-workflow	"Extract & Save Deadlines", called from the Main workflow and from the Web API for every input type.
Error handling and retries	Retry On Fail on Supabase, AI and Telegram download nodes (3 tries, 2 s apart; Claude file reading 2 tries, 3 s apart). Error outputs give the user a clear message. An Error Workflow (Error Trigger) is linked in every workflow's settings and sends me the workflow name, node, error and execution link.